Reinforcement Learning-Driven Evolutionary Multitasking for Tri-Objective Distributed Reentrant Hybrid Flow Shop Scheduling under Speed Scaling

This is the supplementary material of the paper "Reinforcement Learning-Driven Evolutionary Multitasking for Tri-Objective Distributed Reentrant Hybrid Flow Shop Scheduling under Speed Scaling" submitted to the **IEEE Transactions on Cybernetics**.

I. Mathematical Modeling

The notation and constraints of the mixed-integer linear programming (MILP) model are formulated as follows.

1) Indices:

j, j_1, j_2: job indices, $j, j_1, j_2 \in \{1, 2, \ldots, n\}$;
s: stage index, $s \in \{1, 2, \ldots, S\}$;
l, l_1, l_2: lap indices, $l, l_1, l_2 \in \{1, 2, \ldots, L\}$;
f: factory index, $f \in \{1, 2, \ldots, F\}$;
k: machine index, $k \in \{1, 2, \ldots, m_s\}$;
r: speed level index, $r \in \{1, 2, \ldots, R\}$;

2) Parameters:

n: number of jobs;
S: number of stages;
L: number of reentrant laps;
F: number of factories;
m_s: number of machines at stage s;
$M_{s,k}$: the kth machine at stage s;
R: number of speed levels;
v_r: actual speed at level r, where $v_r = 0.5 \cdot (r + 1)$;
$O_{j,s,l}$: operation of job j at stage s in lap l;
$P_{j,s,l}$: standard processing time of job j at stage s in lap l;
d_j: due date of job j;
η_j: penalty factor of job j, $\eta_j \in \{2, 1, 0.5\}$;
IE: idle energy consumption per unit time;
PE_r: processing energy consumption per unit time at speed v_r, $PE_r = 4 \cdot (v_r)^2$;
H: a positive constant, where $H = \sum_{j=1}^{n} \sum_{s=1}^{S} \sum_{l=1}^{L} p_{j,s,l}$.

3) Decision variables:

$p_{j,s,l}$: actual processing time of job j at stage s in lap l;
$C_{j,s,l}$: completion time of job j at stage s in lap l;
D_j: tardiness of job j, $D_j = \max(C_{j,S,L} - d_j, 0)$;
$TP_{s,k,f}$: total processing time of machine $M_{s,k}$ in factory f;
$EP_{s,k,f}$: completion time of machine $M_{s,k}$ in factory f;
$IP_{s,k,f}$: idle time of machine $M_{s,k}$ in factory f;
$C_{\max}$: the makespan value;
TWT: total weighted tardiness;
TIE: total idle energy consumption;
TPE: total processing energy consumption;
TEC: total energy consumption;

4) Binary variables:

$Z_{j,f}$: binary variable, equal to 1 if job j is assigned to factory f, and 0 otherwise;
$X_{j,s,l,k,}$: binary variable, equal to 1 if job j at stage s in lap l is processed on machine $M_{s,k}$, and 0 otherwise;
$W_{j,s,l,r}$: binary variable, equal to 1 if job j at stage s in lap l is processed with speed level r, and 0 otherwise;
$Y_{j_1,l_1,j_2,l_2,s}$: binary variable, equal to 1 if the operation of job j_1 in lap l_1 at stage s is processed before that of job j_2 in lap l_2 at the same stage, and 0 otherwise.
u_j: auxiliary binary variable.

1) Operation-related constraints:

$$\sum_{f=1}^{F} Z_{j,f} = 1, \ \forall j \in \{1, \ldots, n\} \tag{S-1}$$

$$\sum_{k=1}^{m_s} X_{j,s,l,k} = 1, \ \forall j \in \{1, \ldots, n\}, \ \forall s \in \{1, \ldots, S\}, \forall l \in \{1, \ldots, L\} \tag{S-2}$$

$$\sum_{r=1}^{R} W_{j,s,l,r} = 1, \ \forall j \in \{1, \ldots, n\}, \ \forall s \in \{1, \ldots, S\}, \forall l \in \{1, \ldots, L\} \tag{S-3}$$

$$p_{j,s,l} = P_{j,s,l} \cdot \sum_{r=1}^{R} \frac{W_{j,s,l,r}}{v_r}, \ \forall j \in \{1, \ldots, n\}, \forall s \in \{1, \ldots, S\}, \forall l \in \{1, \ldots, L\} \tag{S-4}$$

$$Y_{j_1,l_1,j_2,l_2,s} + Y_{j_2,l_2,j_1,l_1,s} = 1, \forall j_1, j_2 \in \{1, \ldots, n\}, \ \forall l_1, l_2 \in \{1, \ldots, L\}, (j_1, l_1) \neq (j_2, l_2), \forall s \in \{1, \ldots, S\} \tag{S-5}$$

$$C_{j,s,l} \geq p_{j,s,l}, \forall j \in \{1, \ldots, n\}, \forall s \in \{1, \ldots, S\}, \forall l \in \{1, \ldots, L\} \tag{S-6}$$

$$C_{j,s,l} \geq C_{j,s-1,l} + p_{j,s,l}, \ \forall j \in \{1, \ldots, n\}, \forall s \in \{2, \ldots, S\}, \forall l \in \{1, \ldots, L\} \tag{S-7}$$

$$C_{j,1,l} \geq C_{j,S,l-1} + p_{j,1,l}, \forall j \in \{1, \ldots, n\}, \forall l \in \{2, \ldots, L\} \tag{S-8}$$

$$C_{j_2,s,l_2} \geq C_{j_1,s,l_1} + p_{j_2,s,l_2} - H \cdot \Big(5 - Z_{j_1,f} - Z_{j_2,f} - X_{j_1,s,l_1,k} - X_{j_2,s,l_2,k} - Y_{j_1,l_1,j_2,l_2,s}\Big), \forall j_1, j_2 \in \{1, \ldots, n\}, \forall l_1, l_2 \in \{1, \ldots, L\}, (j_1, l_1) \neq (j_2, l_2), \forall s \in \{1, \ldots, S\}, \forall k \in \{1, \ldots, m_s\}, \forall f \in \{1, \ldots, F\} \tag{S-9}$$

$$TP = \sum_{s=1}^{S} \sum_{j=1}^{n} \sum_{l=1}^{L} p_{j,s,l} \tag{S-10}$$

Eq. (S-1) ensures that each operation is assigned to exactly one factory. Eq. (S-2) determines the machine assignment for each operation, while Eq. (S-3) guarantees that exactly one processing speed level is selected. Eq. (S-4) calculates the corresponding processing time. Eq. (S-5) defines the disjunctive relationship between any two distinct operations processed at the same stage. Eqs. (S-6)-(S-8) enforce that a job can proceed to the next stage or reentrant lap only after completing the preceding stage or lap. Eq. (S-9) prevents processing overlap among operations assigned to the same machine in the same factory. Eq. (S-10) calculates the total processing time.

2) Machine-related constraints:

$$EP_{s,k,f} \geq C_{j,s,l} - H \cdot \left(2 - Z_{j,f} - X_{j,s,l,k}\right), \forall j \in \{1,\ldots,n\},$$
$$\forall s \in \{1,\ldots,S\}, \forall k \in \{1,\ldots,m_s\}, \forall l \in \{1,\ldots,L\}, \forall f \in \{1,\ldots,F\} \quad \text{(S-11)}$$

$$IP = \sum_{f=1}^{F} \sum_{s=1}^{S} \sum_{k=1}^{m_s} EP_{s,k,f} - TP \quad \text{(S-12)}$$

Eq. (S-11) determines the final completion time of each machine, while Eq. (S-12) computes the total machine idle time.

3) Objective-related constraints:

$$C_{\max} \geq C_{j,S,L}, \forall j \in \{1,\ldots,n\} \quad \text{(S-13)}$$

$$D_j \geq 0, \forall j \in \{1,\ldots,n\} \quad \text{(S-14)}$$

$$D_j \geq C_{j,S,L} - d_j, \forall j \in \{1,\ldots,n\} \quad \text{(S-15)}$$

$$D_j \leq H \cdot u_j, \forall j \in \{1,\ldots,n\} \quad \text{(S-16)}$$

$$D_j \leq C_{j,S,L} - d_j + H \cdot (1 - u_j), \forall j \in \{1,\ldots,n\} \quad \text{(S-17)}$$

$$TWT = \sum_{j=1}^{n} \eta_j \cdot D_j \quad \text{(S-18)}$$

$$TPE = \sum_{s=1}^{S} \sum_{j=1}^{n} \sum_{l=1}^{L} \left(P_{j,s,l} \cdot \sum_{r=1}^{R} \frac{PE_r}{v_r} \cdot W_{j,s,l,r} \right) \quad \text{(S-19)}$$

$$TIE = IE \cdot IP \quad \text{(S-20)}$$

$$TEC = TIE + TPE \quad \text{(S-21)}$$

Eq. (S-13) defines the makespan. Eqs. (S-14)-(S-18) formulate job tardiness and TWT. Eqs. (S-19)-(S-21) calculate the processing energy, idle energy, and TEC, respectively.

II. Illustrative Example

To illustrate the TDRHFSP-S model, a representative instance is constructed with $R = 6$, $n = 4$, $S = 2$, $L = 2$, $F = 2$, $m_1 = 1$, and $m_2 = 2$. The speed, energy, and job-related processing parameters are provided in Tables S-I and S-II. A feasible schedule X is obtained by assigning one to the selected decision variables in Table S-III and zero to all others, and its corresponding Gantt chart is shown in Fig. S-1. The resulting schedule yields a makespan of $C_{max} = 18$. Among the four jobs, only job 4 is tardy, with a completion time of 15 and a due date of 14.5, yielding $TWT = 1 \times (15 - 14.5) = 0.5$. According to Eq. (S-21), the total energy consumption consists of processing and idle energy. Under the selected speed levels, the processing energy is 684 and the idle energy is 34 with unit idle power $IE = 1$. Therefore, the total energy consumption is $TEC = 684 + 34 = 718$.

TABLE S-I
Speed and Energy Consumption Parameters.

r	1	2	3	4	5	6
v_r	1.0	1.5	2.0	2.5	3.0	3.5
PE_r	$4 \times (v_r)^2$					

III. Propose Approach

For clarity, the flowchart of RLEMTO is presented in Fig. S-2.

TABLE S-II
Job Processing Parameters.

Job	Operations $O_{j,s,l}(P_{j,s,l})$				d_j	η_j
1	$O_{1,1,1}(3)$	$O_{1,2,1}(6)$	$O_{1,1,2}(6)$	$O_{1,2,2}(4)$	15	1.0
2	$O_{2,1,1}(10)$	$O_{2,2,1}(6)$	$O_{2,1,2}(6)$	$O_{2,2,2}(3)$	20	0.5
3	$O_{3,1,1}(2)$	$O_{3,2,1}(5)$	$O_{3,1,2}(9)$	$O_{3,2,2}(6)$	13	2.0
4	$O_{4,1,1}(6)$	$O_{4,2,1}(6)$	$O_{4,1,2}(6)$	$O_{4,2,2}(7)$	14.5	1.0

TABLE S-III
Decision Variables Equal to One in the Feasible Schedule.

Category	Variables
Factory assignment	$Z_{1,1}, Z_{2,1}, Z_{3,2}, Z_{4,2}$
Machine assignment	$X_{1,1,1,1}, X_{2,1,1,1}, X_{1,1,2,1}, X_{2,1,2,1},$ $X_{1,2,1,1}, X_{1,2,2,1}, X_{2,2,1,2}, X_{2,2,2,2},$ $X_{3,1,1,1}, X_{4,1,1,1}, X_{3,1,2,1}, X_{4,1,2,1},$ $X_{3,2,1,1}, X_{3,2,2,1}, X_{4,2,1,2}, X_{4,2,2,2}.$
Speed selection	$W_{1,1,1,2}, W_{2,1,1,3}, W_{1,1,2,2}, W_{2,1,2,2},$ $W_{1,2,1,3}, W_{1,2,2,1}, W_{2,2,1,2}, W_{2,2,2,1},$ $W_{3,1,1,1}, W_{4,1,1,2}, W_{3,1,2,5}, W_{4,1,2,3},$ $W_{3,2,1,1}, W_{3,2,2,3}, W_{4,2,1,2}, W_{4,2,2,6}.$
Processing order on the machine	$Y_{1,1,2,1,1}, Y_{1,1,1,2,1}, Y_{1,1,2,2,1}, Y_{2,1,1,2,1},$ $Y_{2,1,2,2,1}, Y_{1,2,2,2,1}, Y_{1,1,1,2,2}, Y_{2,1,2,2,2},$ $Y_{3,1,4,1,1}, Y_{3,1,3,2,1}, Y_{3,1,4,2,1}, Y_{4,1,3,2,1},$ $Y_{4,1,4,2,1}, Y_{3,2,4,2,1}, Y_{3,1,3,2,2}, Y_{4,1,4,2,2}.$

A. Genetic Operations

A unified genetic framework is applied to all task-specific subpopulations to enhance search efficiency. Taking the main task population P^{main} as an example, parent solutions are selected via dominance-based binary tournament selection; if two solutions are non-dominated, the one closer to the ideal point [1] is preferred. For offspring generation, crossover is performed with probability P_c, where the precedence operation crossover (POX) [2] is applied to JS to preserve job-order information, and the uniform crossover (UX) [3] is applied to FA and SV to recombine genetic information. Mutation is performed with probability P_m, employing random position swap (RPS) to JS, random factory reset (RFR) to FA, and random speed-level replacement (RSR) to SV. The detailed genetic operations are illustrated in Fig. S-3.

B. Knowledge-guided Speed Adjustment

To further exploit the optimization potential of speed decisions, a knowledge-guided speed adjustment mechanism is developed. It exploits critical-path information, operation slack, and job tardiness knowledge extracted from decoded schedules to guide hierarchical speed adjustment. Specifically, it applies three speed adjustment strategies targeting TEC, $C_{\max}$, and TWT, expanding the search within the feasible speed space.

1) Decelerate non-critical operations: First, critical-path analysis identifies non-critical operations. Their speeds are then progressively reduced to the lowest feasible level—without increasing makespan or violating feasibility—to minimize TEC. To prevent TWT degradation, a due-date constraint

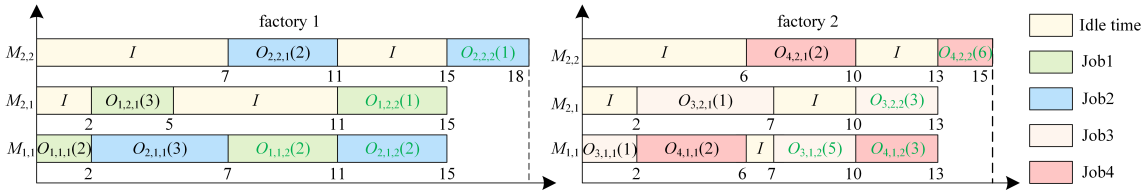


Fig. S-1. Gantt chart of the scheduling solution (different colors represent different jobs, and pale-yellow segments denote machine idle periods. Multiple blocks of the same color correspond to different operations of the same job. Each operation is labeled as $O_{j,s,l}(r)$, where r denotes the speed level. Second-lap operations are highlighted in green).

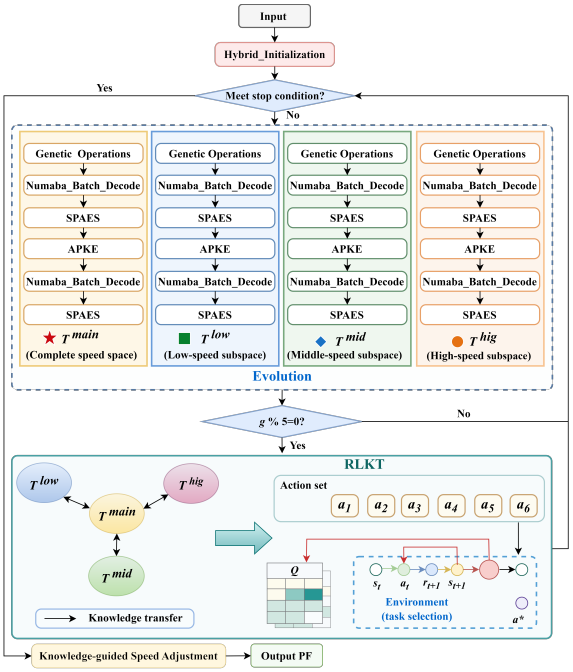


Fig. S-2. The flowchart of the RLEMTO.

applies to each job's last operation: deceleration is blocked if its completion time already exceeds the due date; otherwise, speed reverts to original upon due-date violation. As illustrated in Fig. S-4(a), operations $O_{3,1,1}$, $O_{4,1,1}$, $O_{3,1,2}$, $O_{4,1,2}$, $O_{3,2,1}$, and $O_{4,2,1}$ in factory 2 retain their original speeds (deceleration would delay successors), while $O_{1,1,2}$, $O_{3,2,2}$ and $O_{4,2,2}$ are constrained by the due dates. Only $O_{1,2,1}$ is decelerated from level 3 to level 2, cutting TEC by 13 units.

2) Accelerate critical operations: All critical operations are increased by one speed level to shorten the critical path and reduce makespan. Fig. S-4(b) shows that accelerating $O_{1,1,1}$, $O_{2,1,1}$, $O_{1,1,2}$, $O_{2,1,2}$, $O_{2,2,1}$, and $O_{2,2,2}$ in factory 1 reduces its completion time from 18 to 13.5. As factory 2 finishes at 15, the overall makespan decreases from 18 to 15.

3) Accelerate tardy operations: Finally, operations in tardy jobs are raised by one speed level to reduce TWT. As illustrated in Fig. S-4(c), speeding up $O_{4,1,1}$, $O_{4,2,1}$, and $O_{4,1,2}$ in factory 2 reduces TWT by 0.5 units.

Notably, the knowledge-guided speed adjustment modify only the SV without changing the JS or FA. Offspring are evaluated via *Numba_batch_decoding* using Algorithm 3, generating three distinct offspring per individual: energy-saving, critical-path acceleration, and tardy-job acceleration solutions.

IV. Experimental Results and Analysis

A. Performance Metrics

HV measures the Lebesgue volume $\Lambda(\cdot)$ of the objective space dominated by PF_i with respect to a predefined reference point r:

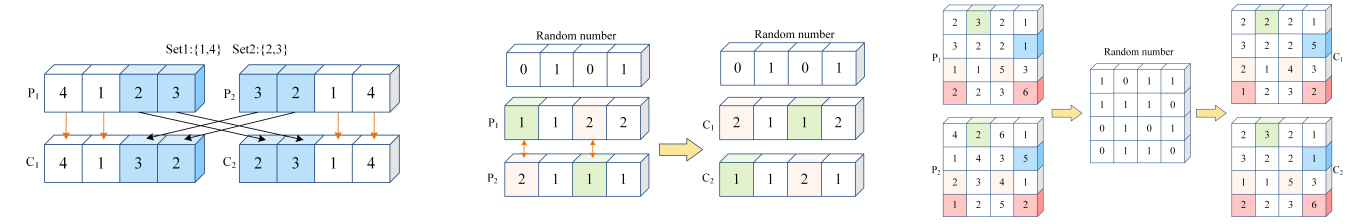
$$HV(PF_i, r) = \Lambda\left(\bigcup_{x \in PF_i} \prod_{m=1}^{3} [\hat{f}_m(x), r_m]\right). \tag{S-22}$$

where $r = (1.1, 1.1, 1.1)$ denotes the reference point. All objective values are normalized using min–max normalization, with the minimum and maximum values determined from the reference front constructed by combining the non-dominated solutions obtained by all algorithms. A larger HV value indicates better convergence and diversity of the obtained solution set.

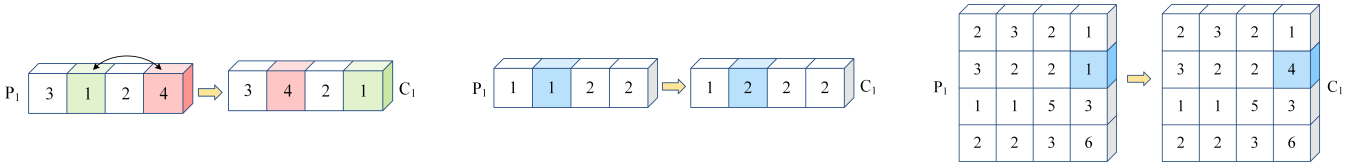
B. Parameters Setting

RLEMTO involves seven key parameters: the main task population size (N^{main}), auxiliary task population size (N^{low}, N^{mid}, N^{hig}), neighborhood radius (R_n), learning rate (α), discount factor (γ), the crossover rate (p_c), and the mutation rate (p_m). Following [4], p_c and p_m are fixed at 0.8 and 0.1, respectively, while the remaining five parameters are calibrated using the Taguchi method [5]. Four levels are assigned to each parameter: $N^{main} \in \{50, 100, 150, 200\}$, $N^{low} = N^{mid} = N^{hig} \in \{15, 20, 25, 30\}$, $R_n \in \{0.05, 0.1, 0.15, 0.2\}$, $\alpha \in \{0.1, 0.2, 0.3, 0.4\}$, and $\gamma \in \{0.6, 0.7, 0.8, 0.9\}$. An $L_{16}(4^5)$ orthogonal array is adopted to generate 16 parameter configurations. Fig. S-5 illustrates the main effects of these parameters on IGD and HV.

The experimental results reveal clear trade-offs among the five parameters. Larger N^{main}, N^{low}, N^{mid}, and N^{hig} retain more high-quality solutions and provide richer auxiliary knowledge, respectively, at the cost of increased computational overhead. The neighborhood radius R_n balances convergence and diversity: a larger R_n weakens the discrimination among representative solutions, whereas a smaller R_n may reduce population diversity. The learning rate α controls the adaptation speed of the transfer policy. A larger α accelerates learning but may cause unstable policy updates, whereas a smaller value slows convergence. The discount factor γ balances immediate and long-term rewards. A larger γ favors globally effective policies, whereas a smaller γ tends to

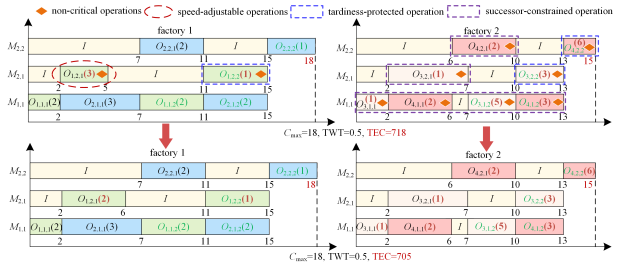


(a) POX for JS. (b) UX for FA. (c) UX for SV.

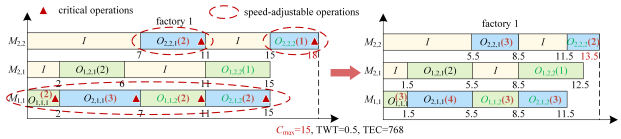


(d) RPS for JS. (e) RFR for FA. (f) RSR for SV.

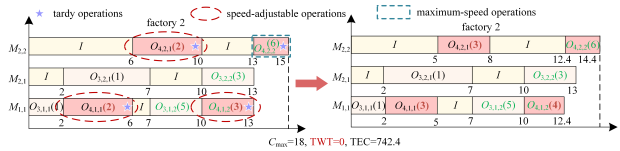
Fig. S-3. Procedures of crossover and mutation operators.



(a) Deceleration of non-critical operations.

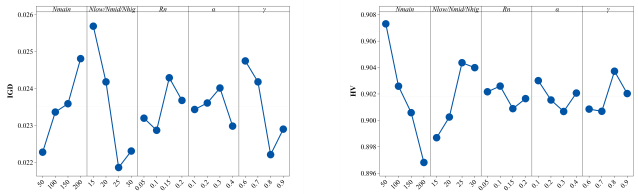


(b) Acceleration of critical-operations.



(c) Acceleration of tardy operations.

Fig. S-4. Illustration of the KNS.



(a) IGD. (b) HV.

Fig. S-5. Main effects plots of the five parameters.

overemphasize immediate rewards and increases the risk of premature convergence. Considering both solution quality and computational efficiency, the parameter configuration $N^{main} = 50$, $N^{low} = N^{mid} = N^{hig} = 25$, $R_n = 0.1$, $\alpha = 0.1$, and $\gamma = 0.8$ is adopted throughout the paper.

C. Model Validation

To validate the proposed MILP model and evaluate the performance of RLEMTO, six benchmark instances are generated. The maximum solving time of Gurobi is set to 3600 s, whereas the termination criterion of RLEMTO is $t_{\max}$ s. Table S-IV summarizes the comparison between Gurobi and RLEMTO in terms of IGD, HV, computational time, and the optimality gap of the MILP model.

For relatively small instances (e.g., R6n5S2L1F2, R7n3S22F2, and R8n5S5L1F2), although Gurobi produces more nondominated solutions, they are concentrated in limited regions of the PF, leading to inferior IGD and HV values. By contrast, RLEMTO achieves better convergence and diversity, resulting in higher-quality PFs. As the problem size increases (e.g., R7n10S5L2F2, R8n10S10L3F2, and R6n20S8L4F2), the computational complexity of the MILP model increases rapidly, preventing Gurobi from obtaining high-quality solutions within the prescribed time limit. Consequently, the optimality gap widens, and both IGD and HV deteriorate significantly. In contrast, RLEMTO consistently generates well-converged and well-distributed PFs at substantially lower computational cost, highlighting its effectiveness and scalability for large-scale TDRHFSP-S instances.

D. Validation of the EMTO Framework

To further assess the scalability of the proposed method, Fig. S-6 presents performance comparisons of RLEMTO and its task ablation variants across different instance scales, characterized by R, F, L, n, and S. Fig. S-7 illustrates the three-dimensional PFs and corresponding two-

TABLE S-IV
COMPARISON OF MILP AND RLEMTO.

Instance	MILP				RLEMTO		
	IGD	HV	Time(s)	Gap(%)	IGD	HV	Time(s)
R6n5S2L1F2	0.31	0.18	4.39	**0.00**	**0.08**	**0.94**	**0.61**
R7n3S22F2	0.59	0.14	6.46	**0.00**	**0.09**	**0.84**	**0.74**
R8n5S5L1F2	0.31	0.28	808.36	**0.00**	**0.07**	**0.88**	**1.54**
R7n10S5L2F2	0.33	0.25	3600	4.37	**0.07**	**0.91**	**6.07**
R8n10S10L3F2	0.31	0.26	3600	4.36	**0.08**	**0.87**	**6.03**
R6n20S8L4F2	0.83	0.13	3600	60.81	**0.03**	**0.60**	**38.62**

TABLE S-V
PARAMETERS OF THE CASE STUDY.

F	n	L	S	R	m_1	m_2	m_3	m_4	m_5	m_6
2	5	5	6	6	3	2	4	2	2	3

dimensional projections of the four evolutionary tasks on instance V8n25S10L3F5.

E. Validation of the Strategy Modules

To further evaluate the scalability of the proposed method, additional performance comparisons of RLEMTO and its component ablation variants across different instance scales, characterized by R, F, L, n, and S, are presented in Fig. S-8.

F. Comparison Experiments and Analysis

To further evaluate the scalability of RLEMTO, Fig. S-9 presents additional performance comparisons with the competing algorithms across different instance scales, characterized by R, F, L, n, and S.

G. Case Study

The detailed parameters, processing times, and due-date information used in the case study are provided in Tables S-V, S-VI, and S-VII, respectively.

H. Discussion

To further investigate the applicability of Numba-based batch decoding, Fig. S-10 compares the numbers of iterations completed by different algorithms with and without *Numba_Batch_Decode* under the same time budget. As observed, the acceleration provided by Numba is algorithm-dependent rather than universal. The results indicate that Numba-based batch decoding is most effective for evolutionary algorithms in which objective evaluation dominates the computational cost and a large number of homogeneous candidate solutions can be evaluated simultaneously. Under such conditions, batch decoding effectively amortizes Python interpreter overhead and improves numerical execution efficiency. In contrast, when the runtime is dominated by control-intensive operations, such as nondominated sorting, archive maintenance, and population management, the overall speedup becomes limited because these operations involve irregular branching and object manipulation. Therefore, Numba-based batch decoding is particularly suitable for evaluation-intensive evolutionary algorithms. The computational workflow of RLEMTO naturally matches these characteristics, leading to notable computational acceleration.

Although RLEMTO demonstrates superior performance on the TDRHFSP-S, its computational efficiency is still constrained by control-intensive modules. Future work will investigate heterogeneous parallel acceleration for key components, including environmental selection, nondominated sorting, and population management, and extend RLEMTO to dynamic, uncertain, and larger-scale scheduling problems to further improve its computational efficiency, scalability, and practical applicability.

TABLE S-VI
PRODUCT PROCESSING INFORMATION.

	Stage 1	Stage 2	Stage 3
Lap 1	8/6/11/14/8	9/10/10/9/7	23/20/19/23/23
Lap 2	0/6/0/14/8	12/0/0/12/11	18/20/19/18/18
Lap 3	0/6/11/0/0	12/0/0/12/11	18/20/19/18/18
Lap 4	8/0/11/0/8	0/13/13/0/0	23/20/19/23/23
Lap 5	8/0/0/14/0	0/13/13/0/7	18/20/19/18/18
	Stage 4	Stage 5	Stage 6
Lap 1	15/14/18/15/16	11/11/10/10/9	5/6/7/5/6
Lap 2	17/0/16/0/14	11/11/10/10/0	5/6/7/5/6
Lap 3	15/14/0/0/16	0/0/10/10/9	5/6/7/5/6
Lap 4	0/14/16/15/0	0/11/0/0/9	5/6/7/5/6
Lap 5	17/16/0/15/14	11/0/10/0/0	5/6/7/5/6

Note: Each entry lists the processing times of Products 1–5 in order, separated by "/". A value of 0 indicates that the corresponding product skips the stage.

TABLE S-VII
DUE DATE INFORMATION OF PRODUCTS.

	Product 1	Product 2	Product 3	Product 4	Product 5
d_j	63	65	35	32	17
η_j	0.5	1	1	2	0.5

REFERENCES

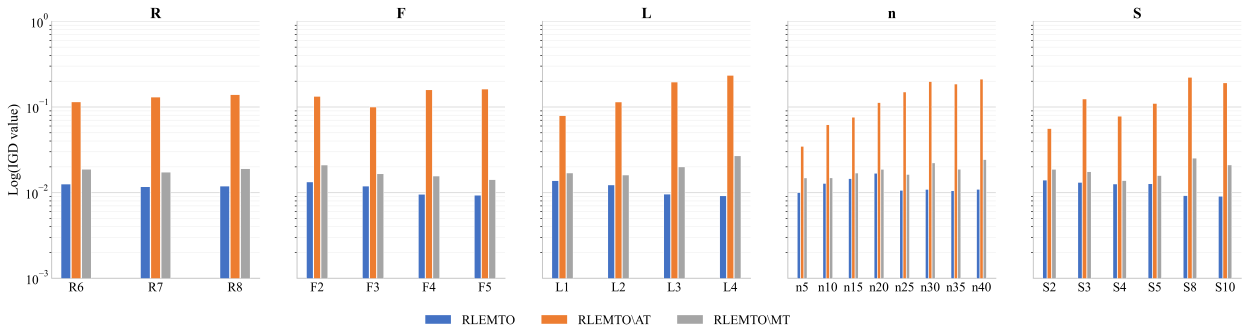
[1] Y. Wang, Y. Han, Y. Wang, Q.-K. Pan, and L. Wang, "Sustainable Scheduling of Distributed Flow Shop Group: A Collaborative Multi-Objective Evolutionary Algorithm Driven by Indicators," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 6, pp. 1794–1808, Dec. 2024.

[2] C. Li, Y. Han, Y. Wang, Q. Chen, and F. Gao, "Clustering and entropy-guided unsupervised learning for multi-objective evolutionary optimization of reentrant hybrid flow shop scheduling," *Swarm and Evolutionary Computation*, vol. 106, p. 102414, Jun. 2026.

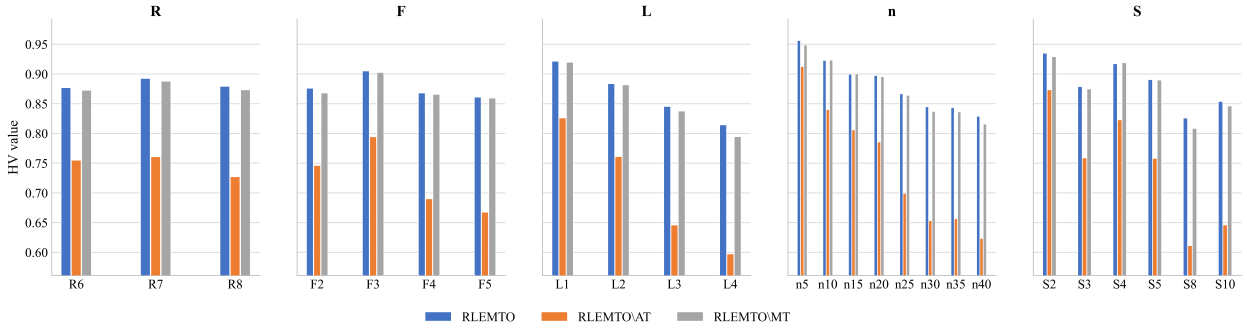
[3] C. Luo, X. Li, W. Gong, and L. Gao, "Affinity Propagation Hierarchical Memetic Algorithm for Multimodal Multi-Objective Flexible Job Shop Scheduling With Variable Speed," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2024.

[4] X. Wu and Z. Xie, "Improved MOEA/D with local search for solving multi-stage distributed reentrant hybrid flow shop scheduling problem," *Expert Systems with Applications*, vol. 232, p. 120893, Dec. 2023.

[5] R. Li, W. Gong, L. Wang, C. Lu, and C. Dong, "Co-Evolution With Deep Reinforcement Learning for Energy-Aware Distributed Heterogeneous Flexible Job Shop Scheduling," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 1, pp. 201–211, Jan. 2024.

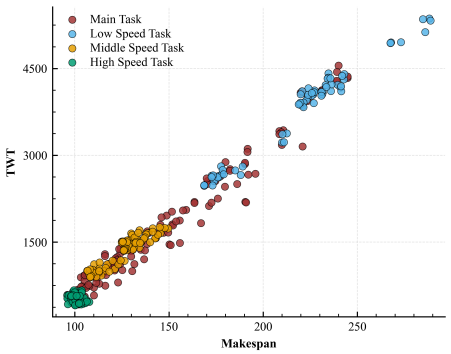


(a) IGD.

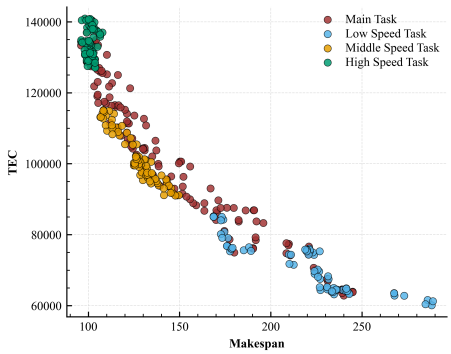


(b) HV.

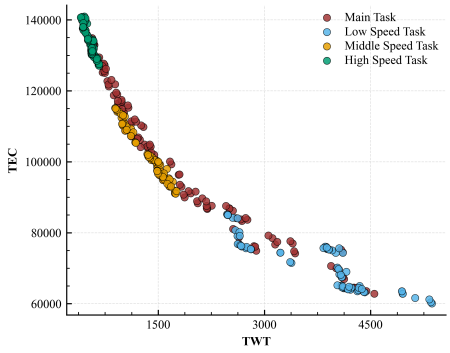
Fig. S-6. Performance of RLEMTO and task ablation variants grouped by instance factors R, F, L, n, and S (The IGD plot uses a logarithmic y-axis, while the HV plot uses a linear y-axis).



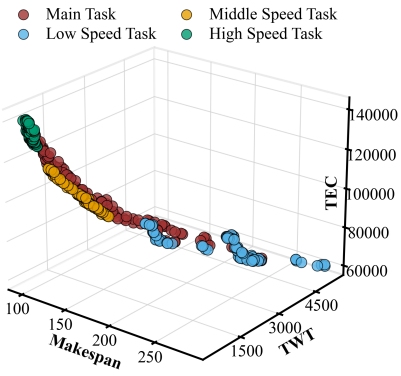
(a) Makespan & TWT.



(b) Makespan & TEC.

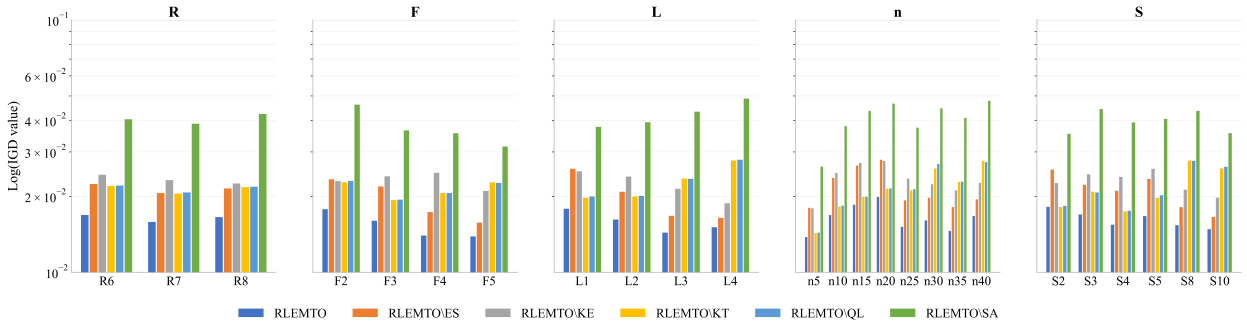


(c) TWT & TEC.

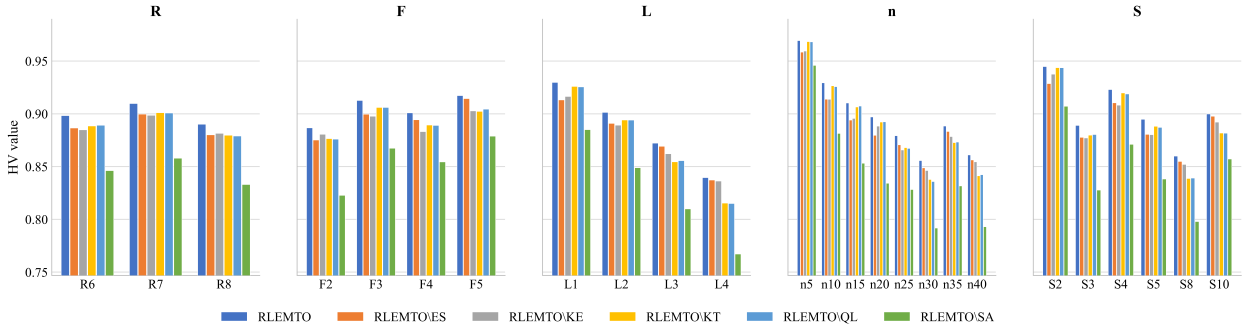


(d) Makespan & TWT & TEC.

Fig. S-7. 2D projections and 3D visualizations of PF for instance V8n25S10L3F5, covering both main and auxiliary tasks.

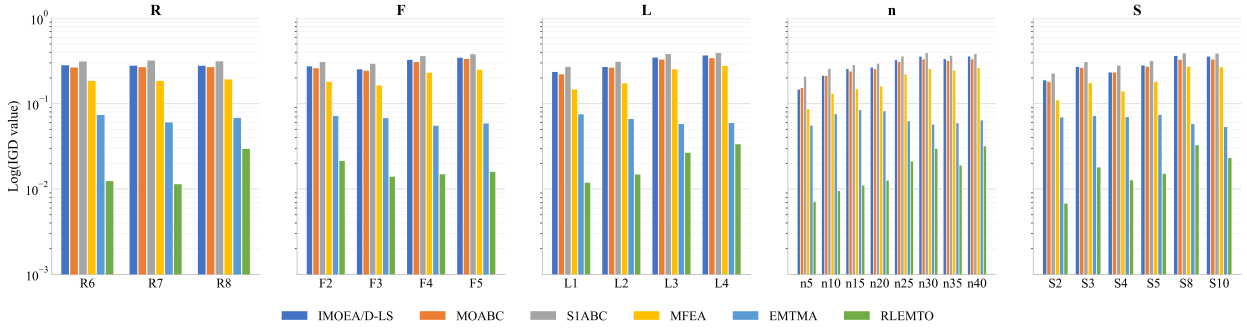


(a) IGD.

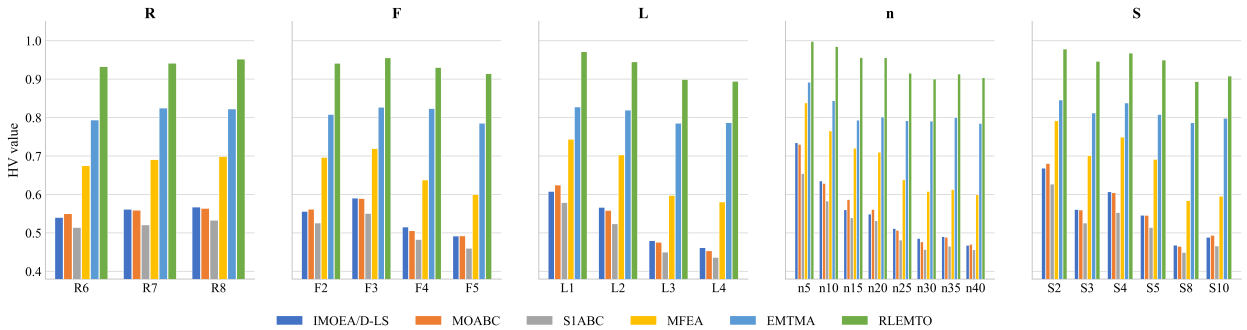


(b) HV.

Fig. S-8. Performance of RLEMTO and component ablation variants grouped by instance factors R, F, L, n, and S (The IGD plot uses a logarithmic y-axis, while the HV plot uses a linear y-axis).



(a) IGD.



(b) HV.

Fig. S-9. Performance of RLEMTO and comparison algorithms grouped by instance factors R, F, L, n, and S (The IGD plot uses a logarithmic y-axis, while the HV plot uses a linear y-axis).

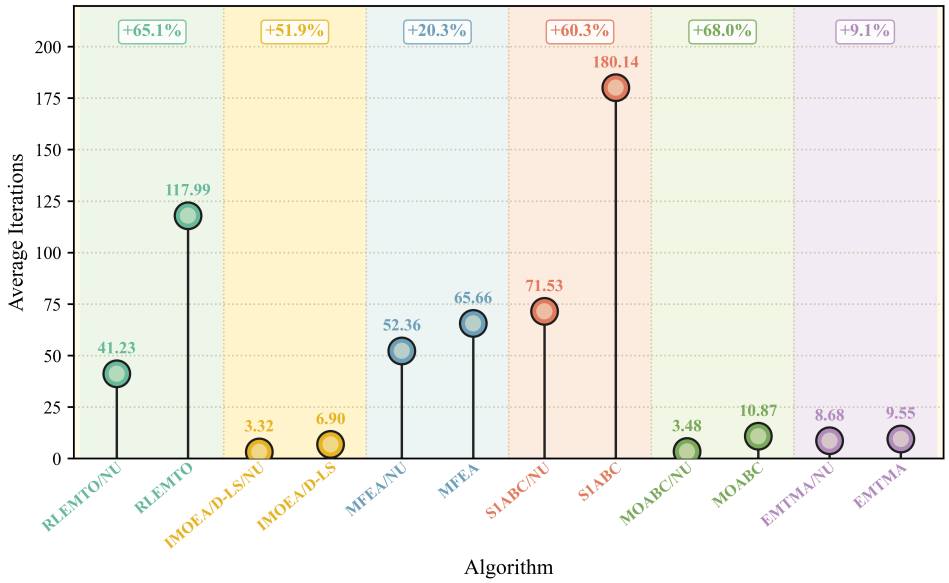


Fig. S-10. Comparison of the number of iterations completed with and without Numba_Batch_Decoding under the same time budget.